

Gassidy: Gaussian Splatting SLAM in Dynamic Environments

Long Wen¹, Shixin Li¹, Yu Zhang¹, Yuhong Huang¹, Jianjie Lin¹,
Fengjunjie Pan¹, Zhenshan Bing^{1,2,3}, and Alois Knoll¹

Abstract—3D Gaussian Splatting (3DGS) allows flexible adjustments to scene representation, enabling continuous optimization of scene quality during dense visual simultaneous localization and mapping (SLAM) in static environments. However, 3DGS faces challenges in handling environmental disturbances from dynamic objects with irregular movement, leading to degradation in both camera tracking accuracy and map reconstruction quality. To address this challenge, we develop an RGB-D dense SLAM which is called Gaussian Splatting SLAM in Dynamic Environments (Gassidy). This approach calculates Gaussians to generate rendering loss flows for each environmental component based on a designed photometric-geometric loss function. To distinguish and filter environmental disturbances, we iteratively analyze rendering loss flows to detect features characterized by changes in loss values between dynamic objects and static components. This process ensures a clean environment for accurate scene reconstruction. Compared to state-of-the-art SLAM methods, experimental results on open datasets show that Gassidy improves camera tracking precision by up to 97.9% and enhances map quality by up to 6%. Video of experiments is available here: <https://www.wixsite.com.com/wen-Gassidy>.

I. INTRODUCTION

Dense visual simultaneous localization and mapping (SLAM), known for its ability to present complex environments, is commonly used in tasks like mobile robot navigation [1]–[3]. These methods rely on known information about static environments to construct accurate maps. However, mobile robots often work in dynamic environments where unpredictable changes can reduce SLAM’s mapping accuracy [4], [5]. Therefore, addressing the challenges posed by dynamic environmental changes is essential to improving the effectiveness of SLAM in mobile robot tasks.

Recently, researchers have integrated Neural Radiance Fields (NeRF) into SLAM to reconstruct scenes in dynamic environments, as it can capture complex lighting effects and fine surface details [6]–[8]. Through generating optical flow and employing semantic segmentation, NeRF-based methods excel at filtering out disturbances of the challenging dynamic environment [9], [10]. However, these methods rely on predefined semantic segmentation to account for dynamic changes, which often fail to capture the irregular movements of objects [9], [10].

3D Gaussian Splatting (3DGS), which constructs Gaussians independently to represent different regions of the scene, has emerged as a promising solution for this issue [11]. This approach allows environmental changes to be flexibly expressed as changes of Gaussians within specific regions, relieving the need for predefined semantic masks

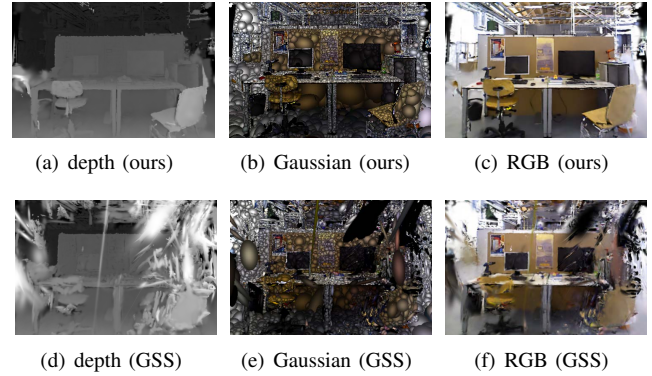


Fig. 1: An example to illustrate the performance of Gassidy when compared with GS-SLAM (here GSS) [15] on the TUM RGB-D dataset in the *fr3/walk_st* scene. The three images in a row represent the rendered depth, the created Gaussians, and the rendered RGB.

[12]–[14]. Despite its advantages, this approach is primarily tailored to handle photometric and geometric changes in static environments and faces challenges in accurately capturing dynamic objects within the scene.

All the aforementioned works focus on recognizing environmental features to reconstruct the scene. However, such approaches either only consider static environments or struggle to effectively handle dynamic environmental changes. The reasons are multi-fold. First, disturbance from dynamic objects often causes overfitting during scene reconstruction, decreasing the accuracy of SLAM. Second, the change in dynamic objects is unpredictable, limiting the application of semantics that rely on prior dynamic knowledge. Third, dynamic objects with minor variations may be incorrectly identified as static environmental components in the viewpoint of subtle movements.

To address these challenges, this paper proposes an optimized 3DGS-based SLAM approach that incorporates rendering loss flows to analyze dynamic environments. We name this method **Gaussian Splatting SLAM in Dynamic environments (Gassidy)**. The approach is designed to filter out disturbances from dynamic objects while tracking the camera pose and reconstructing the scene. An example result of Mapping is illustrated in Fig. 1.

Our contributions are summarized as follows:

- To address unpredictable disturbances from dynamic objects, we use Gaussians to separately cover objects and background features, guided by instance segmentation. Environmental changes caused by dynamic objects are represented as variations in Gaussians rather than relying on predefined semantics.
- To distinguish between Gaussian feature changes caused by dynamic objects and those resulting from photometric or geometric variations in static environments,

¹Technical University of Munich, Germany. {long.wen, zy.zhang, yuhong.huang, jianjie.lin, f.pan, zhenshan.bing, k}@tum.de. ²School of Science and Technology, Nanjing University (Suzhou Campus), China. ³State Key Laboratory for Novel Software Technology, Nanjing University, China
Corresponding author: Zhenshan Bing.

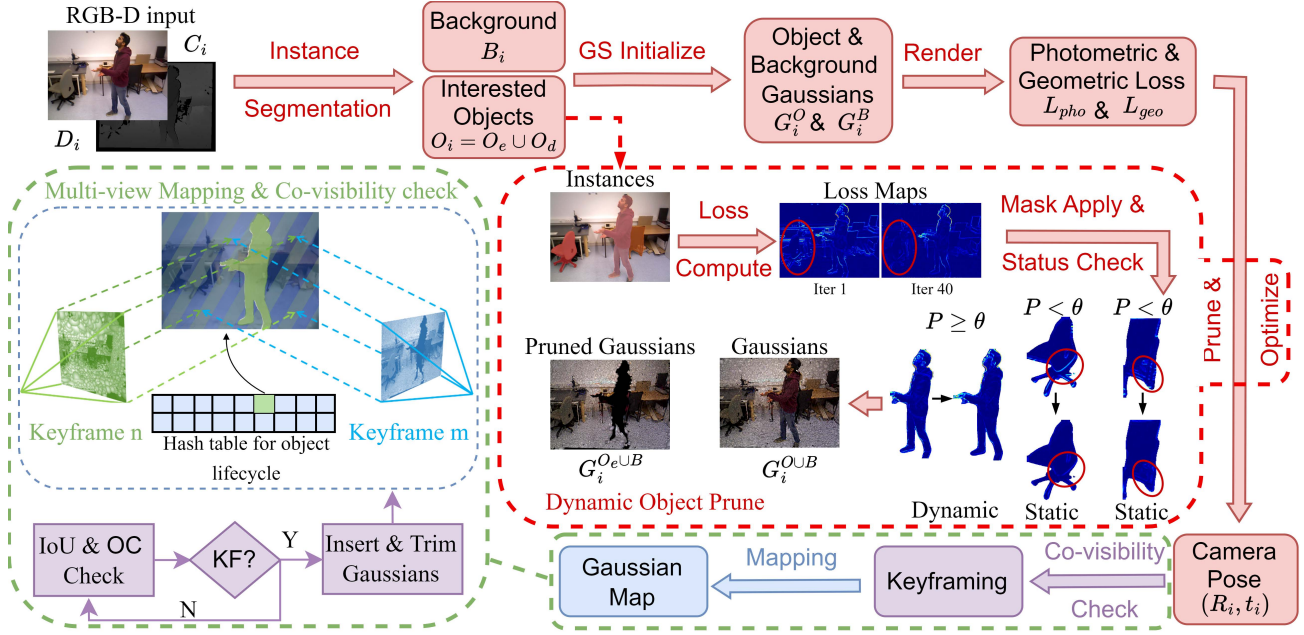


Fig. 2: Architecture of Gassidy: i represents the frame number, C_i and D_i are the RGB image and corresponding aligned depth maps, O_i and B_i are the RGB-D sets of objects and background, G_i^O and G_i^B are the Gaussians of objects and background L_{pho} and L_{geo} are the photometric and geometric rendering loss. R_i and t_i are the rotation and translation parts of the camera pose. IoU and OC stand for Intersection over Union and Overlap Coefficient. G^O and G^{O_e} denote Gaussians for all objects and static objects, while G^B represents background Gaussians. P indicates the probability of an object being dynamic, and θ is the threshold.

Gassidy calculates rendering loss flows for the Gaussians based on a designed photometric-geometric loss function to capture dynamic changes.

- To prevent misidentification of subtle object movements, the rendering loss flows are iteratively calculated to update the camera pose, amplifying the distinction between tiny object changes and subtle frame movements by analyzing features in loss value changes, thereby clearly identifying and filtering dynamic objects.
- Compared to state-of-the-art dense SLAM approaches, Gassidy achieves higher tracking precision of camera pose and reconstructs scenes with finer mapping quality when using widely-used open datasets (“TUM RGB-D” and “BONN Dynamic RGB-D”). In particular, when applying Gassidy, the tracking precision and mapping quality can be enhanced by up to 97.9% and 6%.

II. RELATED WORK

In recent years, the development of autonomous driving and robotics has necessitated detailed environmental maps, which are typically generated using dense visual SLAM methods [16]–[19]. To produce such maps, researchers always assume that the working environment is fixed and does not undergo significant changes during the task. Consequently, SLAM methods generate scenes by treating the environment as static. For example, given fully known environmental information, SplatAM directly optimizes geometric and photometric metrics based on 3D Gaussians to construct the target scene with explicit spatial extent [20]. Similarly, GS-SLAM applies 3DGS to monocular SLAM, utilizing only RGB information to represent environmental features [15]. These methods can precisely reconstruct the scene by analyzing the detailed features of each frame. However, in environments containing dynamic objects, these

methods misidentify the dynamic objects as static, leading to overfitting in scene reconstruction and decreasing the mapping quality.

To handle dynamic environments, researchers aim to filter out dynamic objects by analyzing their movements in detail. Khronos uses spatio-temporal methods to detect dynamic changes utilizing odometry input from other sensors [21]. ONeK-SLAM combines feature points with NeRF to enhance object-level localization and reconstruction in environments with dynamic objects and varying illumination [10]. RoDyn-SLAM enhances dense RGB-D SLAM in dynamic environments through a motion mask generation method and a divide-and-conquer pose optimization algorithm [9]. By predefining the dynamic features of objects, these SLAM approaches can detect and filter out moving objects in dynamic environments, resulting in constructed scenes that align with the target ground truth. However, because these methods rely on predefined semantic priors to model dynamic changes, they often fail to capture unpredictable object movements.

To address this issue, we introduce a novel dense SLAM method based on 3DGS that achieves high-quality mapping in dynamic environments by focusing on variations in the features of constructed Gaussians to distinguish dynamic objects, rather than relying on predefined knowledge.

III. DESIGNING GASSIDY

This section introduces Gassidy’s architecture, details environmental feature extraction using 3DGS, explains filtering dynamic changes via rendering loss flow in tracking, and describes scene optimization with filtered Gaussians.

A. Overview of the Gassidy

With the architecture of Gassidy illustrated in Fig. 2, our objective is to track the camera pose (R_i, t_i) and generate

a clean Gaussian map for scene reconstruction using the input image C_i and its depth information D_i . The tracking process begins by distinguishing objects O_i from the background B_i through the instance masks S_i generated by YOLO segmentation [22]. The set O_i may contain N objects, each assigned an object ID j , with the j^{th} object denoted as $O_i(j)$ for $j \in [0, N]$. Specifically, O_i includes both static and dynamic objects, where the dynamic ones are easily misidentified and need to be filtered out. To minimize reliance on prior environmental knowledge, we initialize Gaussians G_i^O and G_i^B to represent both objects and background without requiring detailed semantics of their dynamic features. Subsequently, we render these Gaussians to compute the rendering loss flows using a photometric-geometric loss function. This process supports us in filtering out dynamic objects and optimizing the camera pose. The detailed procedure is outlined in the “Dynamic Object Prune” section, marked by a red dashed box. The implementation details are provided in the following section.

After filtering out dynamic objects using the loss flows, Gassidy computes an object-level joint loss for optimizing the camera pose. Subsequently, we determine the keyframe that exhibits significant changes compared to the previous one. Once a keyframe is selected, mapping proceeds by constructing the currently visible regions, while rendering loss in pruned areas is excluded from optimization. Pruned regions are reconstructed when subsequent keyframes provide sufficient data. Finally, the features of the Gaussians are updated based on the keyframe, and Gassidy iteratively repeats this process until all images are processed, resulting in a cleanly constructed scene without dynamic object disturbances.

B. Scene representing using 3DGS

To handle unpredictable disturbances from dynamic objects, we utilize 3DGS to represent environmental changes as Gaussian variations across different regions, guided by instance segmentation for potential dynamic object [11]. To apply 3DGS, we first need to perform the Gaussian initialization. The first step of initialization involves converting O_i and B_i into object point clouds $P_i^{O(j)} \in \mathbb{R}^{a_j \times 7}$ and background point clouds $P_i^B \in \mathbb{R}^{b \times 7}$, where a_j is the number of points in the j^{th} object, and b is the number of points in the background. The 7 channels of features in each point cloud consist of 3 RGB channels, 3 coordinate channels, and 1 object ID (j) channel. The coordinates and object ID in $P_i^{O(j)}$ and P_i^B are then used to initialize the Gaussians G_i . Following the method proposed in [11], our Gaussian function $G_i(x)$ is:

$$G_i(x) = e^{-\frac{1}{2}(x-\mu_i)^T(\Sigma_i)^{-1}(x-\mu_i)}, \quad (1)$$

where μ_i is the mean vector representing the position of each Gaussian and Σ_i is the full 3D covariance matrix that defines the spread and orientation of the Gaussian in world space. At this stage, the initial size is set through the scale vector in Σ_i , which has identical values in all three directions, specifically the mean depth value (Z). Additionally, the orientation is initialized with an identity quaternion in Σ_i . Next, we synthesize color and opacity information from the point cloud by splatting and blending the Gaussians, as described in [11]. This process results in the initialization of Gaussians for objects $G_i^{O(j)}$ and the background G_i^B .

Subsequently, we perform rendering to compute the loss by projecting the 3D Gaussians (Σ_W, μ_W) onto 2D space (Σ_I, μ_I) for both objects and background, using the formula:

$$\Sigma_{I,i}^{B \cup O} = J_i^{B \cup O} R_i \Sigma_{W,i}^{B \cup O} R_i^T J_i^{B \cup O T}, \quad (2)$$

where O and B denote the sets of objects and background, $\Sigma_{I,i}^O$ denotes the 2D covariance matrix for the objects in the i^{th} frame of the input RGB image, while $\Sigma_{I,i}^B$ corresponds to the 2D covariance matrix for the background in the same frame. Here, the Jacobian $J_i^{B \cup O}$ approximates the projective transformation for each Gaussian in the background and object set, and R is the rotation matrix derived from the camera pose. The mean μ_I in the 2D space is computed as:

$$\mu_{I,i}^{B \cup O} = \pi(R_i \mu_{W,i}^{B \cup O} + t_i), \quad (3)$$

where π represents the projection operation, and t_i is the translation component of the camera pose. Next, the C_i and D_i (during the initialization, $i = 1$) will be employed as ground truth and start the initial mapping process for enough iterations, updating the features (Σ_i and color) of Gaussians. The loss employed in mapping will be detailed in the next section. After the initial mapping, we obtain a set of fully learned Gaussians that can represent the scene through rendering. Finally, Gassidy proceeds with camera tracking and mapping for subsequent inputs by utilizing the method provided by GS-SLAM, which leverages the Jacobian to guide optimization and minimize the error between the estimated and observed data.

C. Loss define and dynamic objects identification

In this section, we define the loss that will be utilized in tracking and mapping optimization under dynamic environments as well as introduce the logic of dynamic object filtering. Unlike the other 3DGS SLAM, our approach segments the scene into objects of interest and background, utilizing errors from both to optimize the camera pose and mapping accuracy. The loss utilized for optimization in i^{th} frame is:

$$L_{\text{pho}}^{O(j)} = \frac{1}{a_j} \sum_{p \in O(j)} (|\hat{I}_p - I_p| \circ S_{O(j)}), \quad (4a)$$

$$L_{\text{pho}}^B = \frac{1}{b} \sum_{p \in B} (|\hat{I}_p - I_p| \circ \neg \bigcup_{O(j) \in O} S_{O(j)}), \quad (4b)$$

where $L_{\text{pho}}^{O(j)}$ and L_{pho}^B represent the mean photometric rendering loss for the j^{th} object and the background in this frame, respectively, p denote the pixels in the objects or background, $\hat{I}$ is the predicted image through rendering among Gaussians, I is the ground truth image, $S_{O(j)}$ is the mask for the j^{th} object in the frame, and $\circ$ indicates that the mask $S_{O(j)}$ is applied to the loss map. The geometric rendering loss $L_{\text{geo}}^{O(j)}$ and L_{geo}^B are calculated via the identical approach. Additionally, we utilize datasets containing real-world data, where depth values can be unreliable due to inconsistent lighting and sudden changes in highly dynamic environments. To address this challenge, we developed an adaptive loss function that dynamically adjusts the weighting between photometric and geometric rendering losses during

TABLE I: Camera tracking results on dynamic scenes from the **TUM RGB-D** dataset. The best results within each domain are highlighted in **bold**, and the best results among all domains are marked with underline. D/S indicates whether the method is dense or sparse reconstruction, “ATE” column shows the RMSE of the ATE in centimeter, and the “Std.” column presents the standard deviation of ATE. X means tracking failure, and – indicates not mentioned in the original report.

Methods	Type	$f3/wk_xyz$		$f3/wk_hf$		$f3/wk_st$		$f3/st_hf$		Avg.	
Keypoint-based SLAM methods	D/S	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.
ORB-SLAM3 [23]	S	28.1	12.2	30.5	9.0	2.0	1.1	2.6	1.6	15.8	6.0
DynaSLAM [3]	S	<u>1.7</u>	–	2.6	–	0.7	–	2.8	–	2.0	–
NeRF-based SLAM methods	D/S	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.
NICE-SLAM [8]	D	113.8	42.9	X	X	137.3	21.7	93.3	35.3	114.8	33.3
RoDyn-SLAM [9]	D	8.3	5.5	5.6	2.8	1.7	0.9	4.4	2.2	4.1	2.3
3DGS-based SLAM methods	D/S	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.
GS-SLAM [15]	D	37.2	9.9	60.0	20.7	8.4	4.1	7.4	5.4	28.5	10.0
SplaTAM [20]	D	149.2	37.4	157.8	54.4	85.3	16.1	14.0	6.8	125.6	109.6
Gaussian-SLAM [24]	D	133.7	54.8	80.7	31.6	19.1	5.2	5.4	2.2	59.7	23.5
GassiDy (Ours)	D	3.5	1.6	3.7	1.9	<u>0.6</u>	<u>0.3</u>	<u>2.4</u>	1.4	2.6	1.3

TABLE II: Camera tracking results on dynamic scenes from the **BONN Dynamic RGB-D** dataset. The notation is identical to Table I.

Methods	Type	balloon		balloon2		ps_track		ps_track2		mv_box2		Avg.	
Keypoint-based SLAM methods	D/S	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.
ORB-SLAM3 [23]	S	5.8	2.8	17.7	8.6	70.7	32.6	77.9	43.8	3.1	1.6	35.0	17.9
DynaSLAM [3]	S	3.0	–	<u>2.9</u>	–	<u>6.1</u>	–	7.8	–	3.9	–	4.74	–
NeRF-based SLAM methods	D/S	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.
NICE-SLAM [8]	D	X	X	66.8	20.0	54.9	27.5	45.3	17.5	31.9	13.6	49.7	19.7
RoDyn-SLAM [9]	D	7.9	2.7	11.5	6.1	14.5	4.6	13.8	3.5	12.6	4.7	12.1	4.32
3DGS-based SLAM methods	D/S	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.	ATE	Std.
GS-SLAM [15]	D	39.5	19.3	35.6	19.5	93.3	36.3	51.2	19.1	6.1	4.5	45.1	19.7
SplaTAM [20]	D	35.8	14.3	38.7	15.0	138.4	48.1	126.3	36.7	22.0	12.3	54.6	33.7
Gaussian-SLAM [24]	D	65.2	25.5	34.8	22.1	109.2	58.9	118.7	57.2	31.7	20.9	71.9	36.9
GassiDy (Ours)	D	<u>2.6</u>	0.8	7.6	3.4	10.3	4.4	13.0	4.8	5.4	1.9	7.8	3.1

tracking and mapping:

$$L = \lambda_a L_{pho} + (1 - \lambda_a) L_{geo}, \quad (5)$$

where L represents the combined rendering loss used for optimization, L_{pho} and L_{geo} denote the photometric and geometric rendering losses, respectively, and λ_a is an adaptive weight that varies between $[\lambda_{lo}, \lambda_{up}]$ depending on the quality of the depth map. The quality is assessed by the proportion of zero or NaN values within the depth map. As the quality decreases, λ_a increases; we currently model this relationship using a linear function.

We leverage the rendering loss defined in formula 5 for both optimization and dynamic object filtering. To filter out dynamic objects while avoiding the misidentification of subtle object movements, we apply a coarse-to-fine strategy. Initially, we perform 40 iterations of loss computation for both objects and the background, coarsely optimizing the camera pose using only L_{pho}^B and L_{geo}^B , while updating the object loss at each iteration. To track how the loss evolves for static and dynamic objects, we calculate the loss difference between the k^{th} and $(k+1)^{th}$ iterations for $k \in [1, 40]$. After gathering the rendering loss flows over the 40 iterations, we apply a Gaussian Mixture Model (GMM) to classify the background and objects. The key insight is that the loss for the background and static objects decreases consistently over iterations as they become well-aligned with the scene geometry. In contrast, dynamic objects exhibit higher and more fluctuating loss values across iterations due to their motion as shown in Fig. 2. Based on this amplified distinction, we apply the following rule to prune dynamic objects:

$$P(O(j) \in Dynamic | \Delta L^{O(j)}) > \theta \implies O_d \cup O(j), \quad (6)$$

where P is the possibility generated by GMM that $O(j)$ is dynamic, $\Delta L^{O(j)}$ represent the rendering loss flow calculated during iteration for the j^{th} object, the threshold θ determines whether an object should be treated as dynamic, with O_d serving as a set to store dynamic objects. Moreover, we maintain a hash table to manage the life cycle of dynamic objects. For example, if an object is initially classified as static in a single frame, it will not be reintroduced for optimization. Only after being consistently classified as static across three consecutive frames will its information be added back for optimization. Subsequently, all Gaussians with IDs matching those of the objects in O_d are pruned if the last frame is selected as a keyframe. The final object level joint loss L_t used for fine camera pose optimization until it converges is computed as:

$$L_t = L^B + \sum_{O(j) \in O_e} L^{O(j)}, \quad (7)$$

where $O_e = O_d \setminus O$.

After filtering out dynamic objects and accurately estimating the camera pose for the input image, we need to identify the keyframe that provides sufficient new information to update the Gaussians during mapping. Keyframe selection is based on co-visibility checks using Intersection over Union (IoU) and Overlap Coefficient (OC). They are chosen if their IoU is below 80% and their OC exceeds 20%, ensuring comprehensive coverage while filtering out frames with excessive changes. Once the keyframe is determined, we compute the photometric-geometric L_m loss for mapping optimization:

$$L_m = \text{Mean}((\hat{I} - I) \odot -S_{O_d}), \quad (8)$$

where S_{O_d} contains the masks for dynamic objects, managed

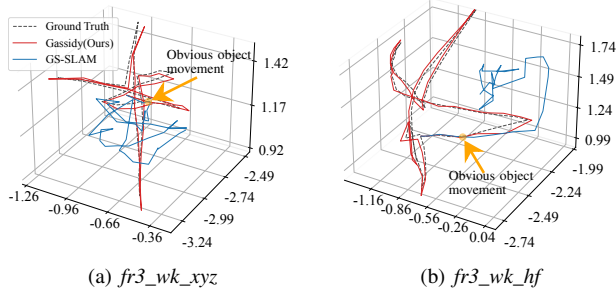


Fig. 3: Camera tracking trajectories of Gassidy and GS-SLAM in dynamic scenes from TUM dataset. Each method’s trajectory, along with the ground truth, is highlighted using different colors, with the moment of significant object movement indicated by an arrow.

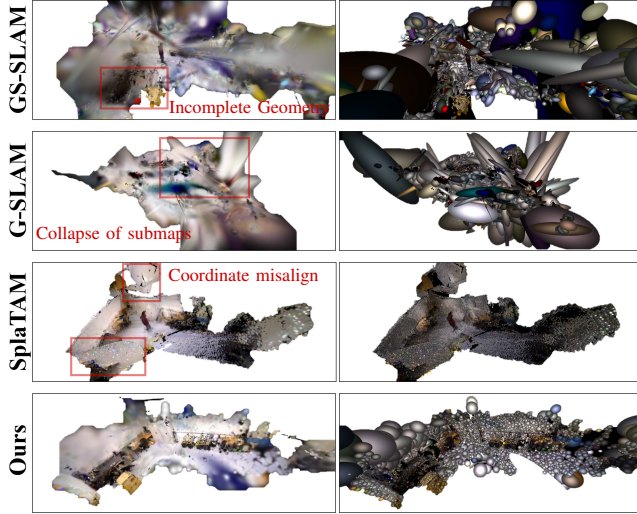


Fig. 4: Large scene reconstruction quality comparison between our method and other 3DGS-based methods in *person_track* scene from BONN dataset. The red box highlights the flaws in these methods.

utilizing the lifecycle method described above. It is updated by referencing the maintained hash table and including dynamic objects still present in the scene. This approach ensures that the loss from dynamic objects does not affect the mapping quality of static components. As illustrated in the section highlighted with a green dashed box in Fig. 2, the Gaussians used for mapping may not cover areas with dynamic objects due to our filtering process during camera tracking. The mapping process will focus on constructing the visible static scene, with Gaussians being added or adjusted when subsequent keyframes provide information about previously empty areas.

IV. EXPERIMENT

To evaluate the proposed Gassidy, we conduct extensive comparison experiments against state-of-the-art SLAM methods. We leverage a variety of metrics to assess camera tracking accuracy and map quality.

A. Experiment Setup

We selected a diverse range of systems, including well-known SLAM methods for static environments [8], [15], [20], [23], [24] and methods optimized for dynamic environments [3], [9]. As no 3DGS-based SLAM methods have been developed for dynamic environments, only those designed

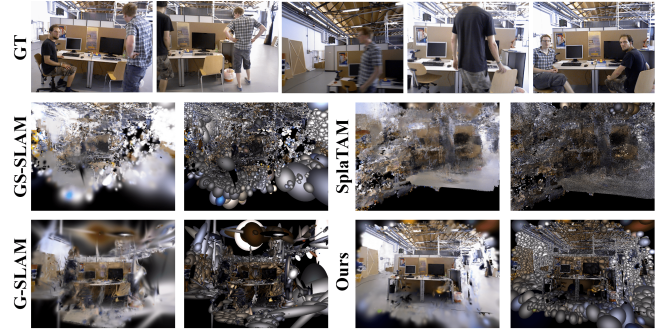


Fig. 5: Map reconstruction quality comparison between our method and other 3DGS-based methods on the *fr3/walk_xyz* scene from TUM dataset. The image on the left shows the rendered RGB, while the image on the right is the generated Gaussian map.

for static environments are included. Our experiments were conducted on two real-world public datasets: TUM RGB-D [25] and BONN RGB-D Dynamic [26]. We selected a variety of dynamic scenes from both datasets, including *fr3/wk_xyz*, *fr3/wk_hf*, *fr3/wk_st*, and *fr3/st_hf* from TUM, as well as *balloon*, *balloon2*, *ps_track*, *ps_track2*, and *mv_box2* from the BONN Dynamic dataset. It is important to note that our benchmarks focus exclusively on 3DGS-based SLAM methods, while results for other domains are sourced from their respective publications. All experiments were conducted on a computer with an Intel i9-12900K CPU and an NVIDIA RTX3080 GPU. The GMM configuration employs an adaptive approach to determine the appropriate number of components ($n_{components}$). Specifically, if the AIC value exceeds 0, the number of components is reduced to 1; otherwise, $n_{components}$ is set to 2. This method effectively mitigates overfitting of GMM, which in this context refers to the misclassification of static objects as dynamic. The adaptive ratio range $[\lambda_{lo}, \lambda_{up}]$ of photometric-geometric loss is in $[0.88, 0.95]$. The threshold θ is set to 99.9%. For point cloud down-sampling, we use a factor of 128/32 for the TUM RGB-D dataset and 256/64 for the BONN dynamic dataset.

For evaluating camera tracking precision, we use the root mean square error (RMSE) and standard deviation (Std.) of the absolute trajectory error (ATE). For map quality evaluation, we employ standard photometric rendering metrics: peak signal-to-noise ratio (PSNR), which indicates image clarity, and structural similarity index measure (SSIM) along with learned perceptual image patch similarity (LPIPS), both of which evaluate the similarity between the input and rendered images. The Map quality metrics are recorded every five frames for detailed analysis.

B. Camera Tracking Precision Analysis

The results for the TUM dataset are presented in Table I. Compared to GS-SLAM, SplaTAM, and Gaussian-SLAM, Gassidy improves the RMSE ATE by an average of 90.9%, 97.9%, and 95.6%, respectively, and enhances the standard deviation by 87.0%, 98.8%, and 94.5%. As a result, we consistently achieve the best performance among 3DGS-based methods. That is because, in those approaches, dynamic objects may be regarded as static objects influenced by slight camera movement, thereby reducing tracking accuracy. In contrast, Gassidy can amplify the difference between static and dynamic objects by iteratively analyzing the features

TABLE III: Map quality results on dynamic scenes in **BONN-
RGBD** dataset. The units are [dB,%,%] for PSNR, SSIM, and LPIPS, respectively. The **best**, **second-best**, and **third-best** value are highlighted in different colors.

Metrics	Scene	GS-SLAM [15]	SplaTam [20]	G-SLAM [24]	Gassidy (Ours)
PSNR _↑	<i>balloon</i>	17.7	17.6	21.6	24.0
	<i>balloon2</i>	19.4	16.8	19.8	22.9
	<i>ps_track</i>	18.9	18.8	24.0	24.6
	<i>ps_track2</i>	20.0	17.5	23.6	24.2
	<i>mv_box2</i>	23.5	20.2	25.1	25.5
SSIM _↑	<i>balloon</i>	71.4	76.9	84.7	77.5
	<i>balloon2</i>	74.8	64.4	77.4	71.5
	<i>ps_track</i>	73.1	68.8	90.6	78.7
	<i>ps_track2</i>	75.6	71.8	89.8	77.3
	<i>mv_box2</i>	83.6	82.7	89.4	85.0
LPIPS _↓	<i>balloon</i>	48.0	24.3	27.5	32.5
	<i>balloon2</i>	36.6	32.6	35.6	39.4
	<i>ps_track</i>	39.6	27.5	20.5	32.8
	<i>ps_track2</i>	37.8	26.3	20.2	32.0
	<i>mv_box2</i>	26.1	18.7	22.3	25.1

of static and dynamic objects, thereby accurately filtering them. In terms of the NeRF-based NICE-SLAM and RoDyn-SLAM, Gassidy shows an average improvement of 97.7% and 36.6% in RMSE ATE, respectively, while also reducing the standard deviation by 96.1% and 43.5%. This is because these approaches depend on detailed semantic segmentation based on prior knowledge. Consequently, they cannot accurately filter out unpredictable objects that are not included in the prior semantics. In contrast, 3DGS can leverage a larger number of input samples for rendering loss computation and optimization, resulting in more detailed information and enhanced performance. Against sparse SLAM methods ORB-SLAM3, Gassidy delivers an average improvement of 83.5% in RMSE ATE and 78.3% in standard deviation. Compared to DynaSLAM, Gassidy improves RMSE ATE by 14.3% in the *f3/wk_st* and *f3/st_hf* scenes. And DynaSLAM shows 25.9% better performance in *f3/wk_xyz* and *f3/wk_hf*. It is noteworthy that although DynaSLAM may attain the highest tracking performance, it consistently neglects environmental details during the mapping process.

The results for the BONN dataset are presented in Table II. The conclusion drawn in the BONN dataset is similar to TUM. Our method substantially outperforms other 3DGS-based methods, achieving improvements of 82.7% in RMSE and 84.3% in standard deviation. When compared to the NeRF-based domain, our method delivers much better performance, with an average improvement of 35.5%. Against DynaSLAM, our method still delivers comparable performance, showing better performance (13.3%) in the *balloon* scene, demonstrating its advanced tracking capabilities. To understand the reasons behind our performance, we visualize some experimental results. As indicated by the red box in Fig. 4, the constructed scenes of compared methods are fragmented. This fragmentation results from overfitting during mapping, causing misalignment with the target coordinate system. In terms of Gassidy, we effectively capture and filter dynamic objects, enabling consistent alignment with the target coordinate system and enhancing camera tracking. Moreover, as shown in Tables I and II, Gassidy consistently achieves the lowest standard deviation, demonstrating its stability. For a detailed analysis, we present the trajectories

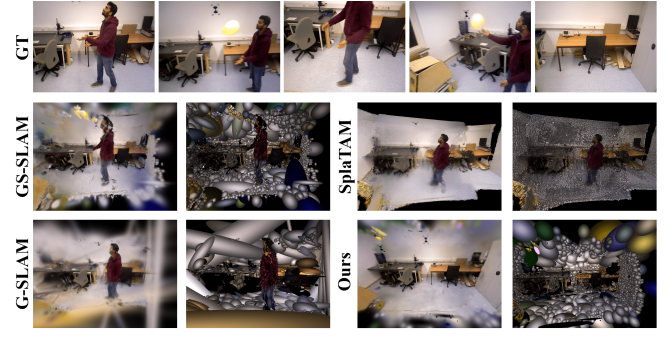


Fig. 6: Map reconstruction quality comparison between our method and other 3DGS-based methods on the *balloon* scene from BONN dataset. The image on the left shows the rendered RGB, while the image on the right is the Gaussian map generated by each method.

for *fr3/wk_xyz* and *fr3/wk_hf* as shown in Fig. 3. Comparison between Gassidy and GS-SLAM (the second-best method) shows that both methods experience reduced tracking accuracy after dynamic object movements. In this case, GS-SLAM struggles to recover due to disturbances from the dynamic objects. In contrast, Gassidy is able to quickly realigns with the ground truth by filtering out dynamic objects, thereby improving the camera tracking precision.

C. Map Quality Analysis

In Table III, the best, second-best, and third-best values are highlighted in distinct colors. It is important to note that the datasets employed lack clear ground truth baselines, so the comparison is based on input images and the results produced by different methods. In this context, as the SSIM and LPIPS reflect the similarity between the input and rendered images, approaches that can filter dynamic objects will result in lower values of these metrics. As illustrated in Figs. 4, 5, and 6, Gassidy, which filters dynamic objects, demonstrates better performance despite lower SSIM and LPIPS values. Moreover, for datasets containing both static and dynamic environments, our approach still achieves acceptable SSIM and LPIPS scores under the influence of the filtered dynamic objects. Furthermore, our method consistently delivers the best PSNR performance, with a 6.0% average improvement over the second-best method. This improvement stems from the fact that disturbances caused by dynamic objects can lead to overfitting during scene reconstruction, resulting in a noisy map and inaccurate camera tracking.

V. CONCLUSIONS

We develop a dense RGB-D SLAM method called Gassidy which leverages a 3D Gaussian representation to handle dynamic environments effectively. To handle the disturbance from irregularly moving objects, we calculate the rendering loss flows for each environment component. By analyzing the loss change features in rendering loss flows, Gassidy distinguishes and filters out the dynamic objects, constructing a high-quality scene with accurate camera tracking. Moreover, our method reduces reliance on semantic priors by requiring only instance segmentation of potential dynamic objects, without needing prior knowledge of their dynamic features. Our future work will focus on enhancing object-level reconstruction and the efficiency of the method for real-time robotics applications.

REFERENCES

- [1] T. Schops, T. Sattler, and M. Pollefeys, “Bad slam: Bundle adjusted direct rgb-d slam,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 134–144.
- [2] A. Dai, M. Nießner, M. Zollhöfer, S. Izadi, and C. Theobalt, “Bundle-fusion: Real-time globally consistent 3d reconstruction using on-the-fly surface reintegration,” *ACM Transactions on Graphics (ToG)*, vol. 36, no. 4, p. 1, 2017.
- [3] B. Bescos, J. M. Fàcil, J. Civera, and J. Neira, “DynaSLAM: Tracking, mapping, and inpainting in dynamic scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018.
- [4] A. Rosinol, J. J. Leonard, and L. Carlone, “Probabilistic volumetric fusion for dense monocular slam,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2023, pp. 3097–3105.
- [5] O. Kähler, V. A. Prisacariu, and D. W. Murray, “Real-time large-scale dense 3d reconstruction with loop closure,” in *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part VIII 14*. Springer, 2016, pp. 500–516.
- [6] M. M. Johari, C. Carta, and F. Fleuret, “Eslam: Efficient dense slam system based on hybrid representation of signed distance fields,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 17 408–17 419.
- [7] H. Wang, J. Wang, and L. Agapito, “Co-slam: Joint coordinate and sparse parametric encodings for neural real-time slam,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2023, pp. 13 293–13 302.
- [8] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, “Nice-slam: Neural implicit scalable encoding for slam,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [9] H. Jiang, Y. Xu, K. Li, J. Feng, and L. Zhang, “Rodyn-slam: Robust dynamic dense rgb-d slam with neural radiance fields,” *IEEE Robotics and Automation Letters*, 2024.
- [10] Y. Z. H. L. R. C. Y. C. J. Yan and Z. Jiang, “Oneslam: A robust object-level dense slam based on jointneural radiance fields and keypoints,” in *IEEE International Conference on Robotics and Automation (ICRA)*, May 2024.
- [11] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.
- [12] S. Sabour, L. Goli, G. Kopanas, M. Matthews, D. Lagun, L. Guibas, A. Jacobson, D. J. Fleet, and A. Tagliasacchi, “Spotlessplats: Ignoring distractors in 3d gaussian splatting,” 2024.
- [13] Q. Gao, Q. Xu, Z. Cao, B. Mildenhall, W. Ma, L. Chen, D. Tang, and U. Neumann, “Gaussianflow: Splatting gaussian dynamics for 4d content creation,” 2024.
- [14] Z. Lu, X. Guo, L. Hui, T. Chen, M. Yang, X. Tang, F. Zhu, and Y. Dai, “3d geometry-aware deformable gaussian splatting for dynamic view synthesis,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [15] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, “Gaussian Splatting SLAM,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [16] L. Zhang, K. Cai, Z. Sun, Z. Bing, C. Wang, L. Figueredo, S. Haddadin, and A. Knoll, “Motion planning for robotics: A review for sampling-based planners,” *Biomimetic Intelligence and Robotics*, vol. 5, no. 1, p. 100207, 2025.
- [17] C. Creß, Z. Bing, and A. C. Knoll, “Intelligent transportation systems using roadside infrastructure: A literature survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 7, pp. 6309–6327, 2024.
- [18] Z. Bing, L. Knak, L. Cheng, F. O. Morin, K. Huang, and A. Knoll, “Meta-reinforcement learning in nonstationary and nonparametric environments,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 10, pp. 13 604–13 618, 2024.
- [19] Y. Zhang, L. Kong, X. Yu, W. He, and A. Knoll, “Robust dual-filter safety control for mobile robots in dynamic multiobstacle environments,” *IEEE/ASME Transactions on Mechatronics*, pp. 1–12, 2025.
- [20] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, “Splatam: Splat, track & map 3d gaussians for dense rgb-d slam,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [21] L. Schmid, M. Abate, Y. Chang, and L. Carlone, “Khronos: A unified approach for spatio-temporal metric-semantic slam in dynamic environments,” in *Proc. of Robotics: Science and Systems*, 2024.
- [22] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [23] C. Campos, R. Elvira, J. J. Gomez, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM3: An accurate open-source library for visual, visual-inertial and multi-map SLAM,” *arXiv preprint arXiv:2007.11898*, 2020.
- [24] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, “Gaussian-slam: Photo-realistic dense slam with gaussian splatting,” 2023.
- [25] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, “A benchmark for the evaluation of rgb-d slam systems,” in *IEEE International Conference on Intelligent Robot Systems (IROS)*, Oct. 2012.
- [26] E. Palazzolo, J. Behley, P. Lottes, P. Giguère, and C. Stachniss, “ReFusion: 3D Reconstruction in Dynamic Environments for RGB-D Cameras Exploiting Residuals,” *arXiv*, 2019. [Online]. Available: <https://arxiv.org/abs/1905.02082>